OPLOGS-08: Security & Data Protection

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 8 of 8 | **Created:** December 2025 | **Last Updated:** 10/12/2026

Sensitive Data Discovery, Masking, and Compliance

This notebook covers sensitive data discovery, OpenPipeline masking configuration, security event monitoring, and compliance reporting.

Table of Contents

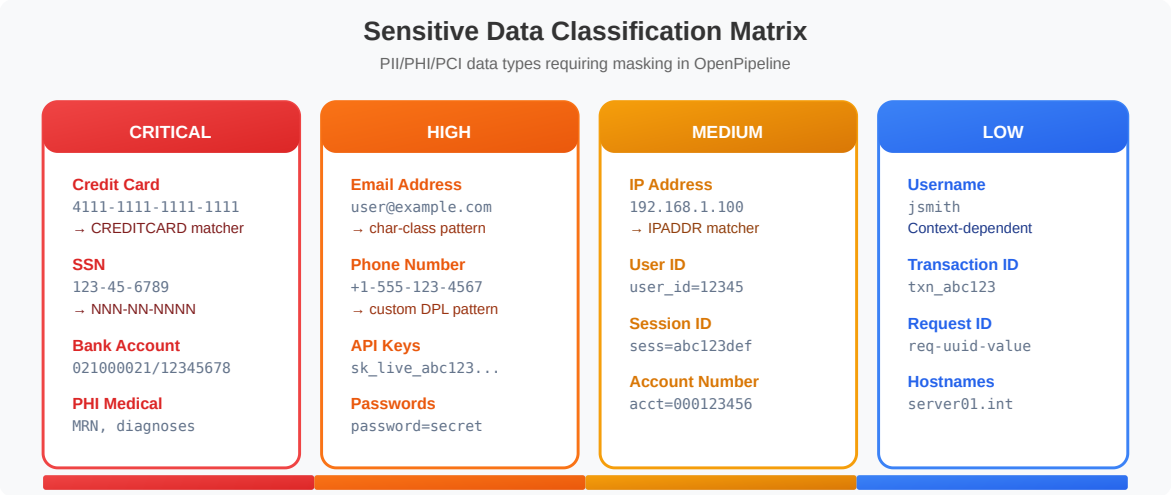
1. Sensitive Data Discovery
2. OpenPipeline Masking Configuration
3. IP Address Analysis
4. Security Event Monitoring
5. Audit Log Queries
6. Compliance Reporting
7. Masking Best Practices
8. 📝 Summary
9. ➡️ Next Steps
10. 📚 References

Prerequisites

- ✅ Access to a Dynatrace environment with log data
- ✅ Completed OPLOGS-01 through OPLOGS-07
- ✅ Understanding of data privacy requirements

1. Sensitive Data Discovery

Before implementing masking, discover what sensitive data exists in your logs.



Common Sensitive Data Patterns

Data Type	Pattern Example	Risk Level
Email addresses	`user@domain.com`	Medium
Credit cards	`4111-1111-1111-1111`	**Critical**
Social Security	`123-45-6789`	**Critical**
API keys	`sk_live_abc123...`	**Critical**
IP addresses	`192.168.1.100`	Low-Medium
Phone numbers	`+1-555-123-4567`	Medium
JWT tokens	`eyJhbG...`	High

```
// Search for potential email addresses in logs
fetch logs, from: now() - 1h
| filter contains(content, "@")
| filter NOT contains(content, "@dynatrace")
| filter NOT contains(content, "@example")
| fieldsAdd content_preview = substring(content, from: 0, to: 150)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 20
```

```
// Search for potential API keys or tokens
fetch logs, from: now() - 1h
| filter contains(content, "key=")
        OR contains(content, "token=")
        OR contains(content, "api_key")
        OR contains(content, "apikey")
        OR contains(content, "secret")
| fieldsAdd content_preview = substring(content, from: 0, to: 120)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 20
```

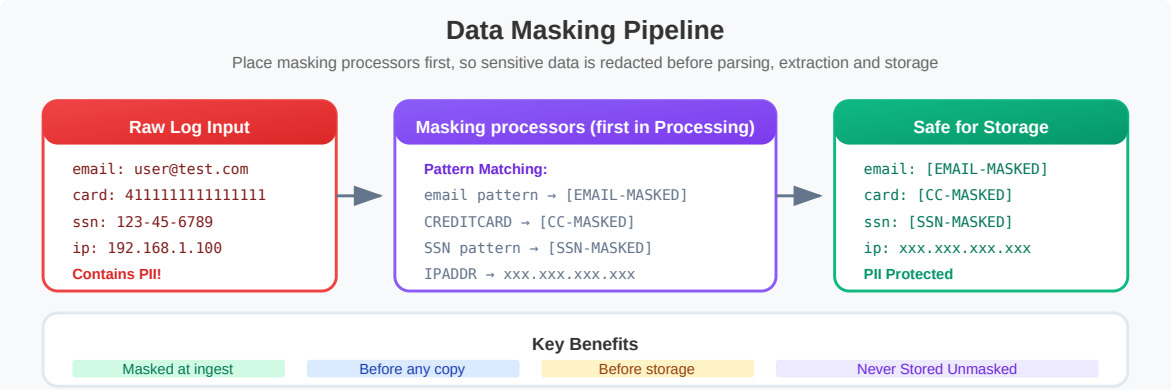
```
// Search for potential JWT tokens
fetch logs, from: now() - 1h
| filter contains(content, "eyJ")  // JWT typically starts with eyJ
| fieldsAdd content_preview = substring(content, from: 0, to: 100)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 15
```

```
// Lines carrying a Luhn-valid card number (CREDITCARD), bounded so digit runs inside UUIDs do not count
// Grouped by source, not by content, so the card numbers are not echoed into the result table
fetch logs, from: now() - 1h
| filter replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD !&gt;&gt; [0-9A-Za-z-]", "") != content
| summarize {count = count()}, by: {k8s.namespace.name, log.source}
| sort count desc
| limit 15
```

```
// Search for password-related entries (should never be logged!)
fetch logs, from: now() - 1h
| filter contains(content, "password")
        OR contains(content, "passwd")
        OR contains(content, "pwd=")
| fieldsAdd content_preview = substring(content, from: 0, to: 100)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 20
```

2. OpenPipeline Masking Configuration

OpenPipeline masks sensitive data at ingestion time with a **DQL processor** in the **Processing** stage — there is no dedicated masking processor.



Masking Options

Processor	Use Case
DQL processor — `replacePattern` / `replaceString` / `ipMask`	Replace a value or part of it (e.g. `[EMAIL-MASKED]`, last octet → 0)
DQL processor — `hashSha256()`	One-way hash when you need to correlate on a value without exposing it
Remove fields	Drop a whole field

Partial masking. `replacePattern` has no back-references, but a `>>` lookahead keeps the trailing digits without one: `replacePattern(content, "!<<[0-9A-Za-z-] [0-9]{3} '-' [0-9]{2} '-' >>([0-9]{4} !>>[0-9A-Za-z-])", "***-**-")` turns `ssn 123-45-6789 rest` into `ssn ***-**-6789 rest`, and `"!<<[0-9A-Za-z-] [0-9]{12} >>([0-9]{4} !>>[0-`

`9A-Za-z-])"` with `"XXXXXXXXXXX"` turns `4111111111111234` into `XXXXXXXXXXX1234`. The `!<<[0-9A-Za-z-]` / `!>>[0-9A-Za-z-]` bounds stop the pattern firing inside a longer number or an alphanumeric ID (digit-only bounds still masked inside `3f2b9c1e4111111111111111aa`) — without them, `order 99999999999999999999` becomes `order XXXXXXXXXXX99999999` (verified with `data record`, 10/12/2026).

Configuration Path

1. Settings → Process and contextualize → OpenPipeline
2. Select or create pipeline
3. Add a **DQL** processor to the **Processing** stage (masking has no dedicated processor)
4. Configure field and pattern

DPL masking statements (DQL processor, Processing stage)

```
fieldsAdd content = replacePattern(content, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "[EMAIL-MASKED]")
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD !&gt;&gt;[0-9A-Za-z-]", "[CC-MASKED]")
fieldsAdd content = replacePattern(content, "[0-9]{3} '-' [0-9]{2} '-' [0-9]{4}", "[SSN-MASKED]")
fieldsAdd content = replacePattern(content, "&lt;&lt;(('api_key' | 'API_KEY' | 'apikey' | 'APIKEY' | 'key' | 'KEY' | 'token' | 'TOKEN') '=') [A-Za-z0-9_-]{20,}", "[KEY-MASKED]")
fieldsAdd content = replacePattern(content, "&lt;&lt;('\"' ('api_key' | 'API_KEY' | 'apikey' | 'APIKEY' | 'key' | 'KEY' | 'token' | 'TOKEN') '\"' ' '* ':' ' '* '\"') [A-Za-z0-9_-]{20,}", "[KEY-MASKED]")
fieldsAdd content = replacePattern(content, "IPADDR", "xxx.xxx.xxx.xxx")
```

> DPL, not regex — see FAQ-15 §3. `CREDITCARD` validates the Luhn checksum. There is no `EMAIL` matcher; the character-class pattern above is the working form. Regex syntax (`\b`, `\d`, `(?:…)`, `$1`) is rejected when you save the processor. Test every statement with `data record(content="…") | fieldsAdd x = replacePattern(…)` before deploying — including against your real key formats. The key class includes `_` and `-`: with `[A-Za-z0-9]{20,}`, a key like `sk_live_51Hx…` (underscores) or a dash-separated key matched nothing and was **stored in clear** (verified 10/02/2026). Keys shorter than 20 characters are not matched by design; lower the bound if yours are shorter. DPL literals are case-sensitive and there is no case-insensitive flag — list each casing your applications emit (`Token=` is not covered above), and keep the

> separate statement for JSON-quoted keys (`"api_key": "…"`). Bound `CREDITCARD` with negative lookarounds: unbounded, it matches Luhn-valid digit runs inside UUIDs (`9cbcb510-8833-4981-87ab-…` → `9cbcb[CC-MASKED]ab-…`) and corrupts the IDs at ingest — on the validation tenant 1,525 lines in 15 minutes matched unbounded and 169 bounded (10/12/2026).
>
> **Sources:** OpenPipeline processing examples (DT docs) — *"You can mask parts of an attribute by leveraging replacePattern in combination with other DQL functions."*

```
// Verify masking is working
// 1) Leak check — unmasked values that the masking patterns would still catch
(should be 0)
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    unmasked_email = countIf(replacePattern(content, "[A-Za-z0-9._%+-]+ '@'
[A-Za-z0-9.-]+", "") != content),
    unmasked_card = countIf(replacePattern(content, "!&lt;&lt;[0-9A-Za-z-]
CREDITCARD !&gt;&gt;[0-9A-Za-z-]", "") != content),
    masked_markers = countIf(contains(content, "MASKED"))
  }
```

```
// Check which pipelines are processing logs
fetch logs, from: now() - 1h
| summarize {count = count()}, by: {dt.openpipeline.pipelines}
| sort count desc
```

3. IP Address Analysis

IP addresses may require masking depending on your compliance requirements.

```
// Find logs containing IP addresses
// parseAll finds every IP anywhere in the line; parse would only match one at
the very start
fetch logs, from: now() - 1h
| fieldsAdd ip_found = parseAll(content, "IPADDR:ip")
| filter arraySize(ip_found) &gt; 0
| expand ip_found
| summarize {count = count()}, by: {ip_found, k8s.namespace.name}
| sort count desc
| limit 20
```

```
// Classify IP addresses (internal vs external)
fetch logs, from: now() - 1h
| fieldsAdd ip_found = parseAll(content, "IPADDR:ip")
| filter arraySize(ip_found) &gt; 0
| expand ip_found
| fieldsAdd ip_type = if(ipIn(ip_found, {"10.0.0.0/8", "172.16.0.0/12",
"192.168.0.0/16"}), "RFC1918",
                     else: if(ipIn(ip_found, "127.0.0.0/8"), "LOOPBACK",
                     else: "EXTERNAL"))
| summarize {count = count()}, by: {ip_type}
| sort count desc
```

4. Security Event Monitoring

Monitor logs for security-relevant events.

```
// Authentication-related logs
fetch logs, from: now() - 1h
| filter contains(content, "login")
        OR contains(content, "auth")
        OR contains(content, "signin")
        OR contains(content, "logout")
// Group on a normalized signature (UUIDs, then digits collapsed); the raw
prefix starts with a timestamp
| fieldsAdd msg_sig = substring(replacePattern(replacePattern(content, "[0-9a-
fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-
fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0,
to: 100)
| summarize {count = count(), sample = takeAny(substring(content, from: 0, to:
100))}, by: {msg_sig, k8s.namespace.name}
| sort count desc
| limit 20
```

```
// Failed authentication attempts
fetch logs, from: now() - 1h
| filter (contains(content, "failed") OR contains(content, "denied") OR
contains(content, "unauthorized"))
        AND (contains(content, "login") OR contains(content, "auth") OR
contains(content, "access"))
| fieldsAdd content_preview = substring(content, from: 0, to: 120)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 20
```

```
// Security-related errors over time
// status == "ERROR" covers every error-class level (ERROR, SEVERE, CRITICAL,
FATAL, …);
// loglevel == "ERROR" alone misses SEVERE and the rest
fetch logs, from: now() - 24h
| filter status == "ERROR"
| filter contains(content, "security")
        OR contains(content, "unauthorized")
        OR contains(content, "forbidden")
        OR contains(content, "denied")
| makeTimeseries {security_errors = count()}, interval: 30m
```

```
// Access pattern anomalies - hourly access distribution
fetch logs, from: now() - 24h
| filter contains(content, "access") OR contains(content, "request")
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {access_count = count()}, by: {hour_bucket, k8s.namespace.name}
| sort hour_bucket asc
```

5. Audit Log Queries

Track administrative and configuration changes.

```
// Administrative action logs
fetch logs, from: now() - 24h
| filter contains(content, "created")
        OR contains(content, "deleted")
        OR contains(content, "updated")
        OR contains(content, "modified")
| filter contains(content, "user")
        OR contains(content, "admin")
        OR contains(content, "config")
// Group on a normalized signature (UUIDs, then digits collapsed); the raw
prefix starts with a timestamp
| fieldsAdd msg_sig = substring(replacePattern(replacePattern(content, "[0-9a-
fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-
fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0,
to: 100)
| summarize {
    count = count(),
    first_seen = min(timestamp),
    last_seen = max(timestamp),
    sample = takeAny(substring(content, from: 0, to: 100))
  }, by: {msg_sig, k8s.namespace.name}
| sort count desc
| limit 20
```

```
// Configuration change events
fetch logs, from: now() - 24h
| filter contains(content, "config")
        OR contains(content, "setting")
        OR contains(content, "environment")
| filter contains(content, "change")
        OR contains(content, "update")
        OR contains(content, "reload")
// Group on a normalized signature (UUIDs, then digits collapsed); the raw
prefix starts with a timestamp
| fieldsAdd msg_sig = substring(replacePattern(replacePattern(content, "[0-9a-
fA-F]{8} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-fA-F]{4} '-' [0-9a-
fA-F]{12}", replacement: "&lt;id&gt;"), "[0-9]+", replacement: "#"), from: 0,
to: 120)
| summarize {count = count(), sample = takeAny(substring(content, from: 0, to:
120))}, by: {msg_sig, k8s.namespace.name}
| sort count desc
| limit 15
```

```
// Permission-related events
fetch logs, from: now() - 24h
| filter contains(content, "permission")
        OR contains(content, "role")
        OR contains(content, "privilege")
        OR contains(content, "rbac")
| fieldsAdd content_preview = substring(content, from: 0, to: 100)
| summarize {count = count()}, by: {content_preview, k8s.namespace.name}
| sort count desc
| limit 15
```

6. Compliance Reporting

```
// Data access summary
fetch logs, from: now() - 24h
| filter contains(content, "access") OR contains(content, "read") OR
contains(content, "view")
| summarize {
    total_accesses = count(),
    unique_sources = countDistinct(dt.openpipeline.source)
  }, by: {k8s.namespace.name}
| sort total_accesses desc
| limit 15
```

```
// Configured retention per log bucket
// (min/max timestamp over a query window cannot verify retention — the window
bounds the answer)
fetch dt.system.buckets
| filter dt.system.table == "logs"
| fields name, retention_days, included_query_days, records
| sort name asc
```

```
// Sensitive field exposure audit
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    with_email_pattern = countIf(contains(content, "@")),
    with_key_pattern = countIf(contains(content, "key=") OR contains(content,
"token=")),
    with_password_ref = countIf(contains(content, "password") OR
contains(content, "passwd"))
  }
| fieldsAdd email_exposure_pct = round((with_email_pattern * 100.0) / total,
decimals: 2)
| fieldsAdd key_exposure_pct = round((with_key_pattern * 100.0) / total,
decimals: 2)
| fieldsAdd password_exposure_pct = round((with_password_ref * 100.0) / total,
decimals: 2)
```

7. Masking Best Practices

Do's ✅

1. **Mask at ingestion** - Use OpenPipeline processors to mask before storage
2. **Use hashing** - For fields that need correlation but not visibility
3. **Audit regularly** - Run discovery queries to find new sensitive data patterns
4. **Document patterns** - Keep a registry of masked data types and patterns
5. **Test in staging** - Verify masking works before production deployment

Don'ts ❌

1. **Don't log passwords** - Ever, even "for debugging"
2. **Don't log full credit cards** - Maximum last 4 digits
3. **Don't rely on application masking alone** - OpenPipeline is your safety net
4. **Don't skip internal data** - Internal IP/usernames may still be sensitive
5. **Don't forget API keys** - Rotate if accidentally logged

OpenPipeline Masking Configuration Example

One **DQL** processor in the **Processing** stage (matcher `true`, or a narrower one), placed before any processor that copies or parses `content`:

```
fieldsAdd content = replacePattern(content, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "[EMAIL-MASKED]")
| fieldsAdd content = replacePattern(content, "&lt;&lt;(('api_key' | 'API_KEY' | 'apikey' | 'APIKEY' | 'key' | 'KEY' | 'token' | 'TOKEN') '=') [A-Za-z0-9_-]{20,}", "[KEY-MASKED]")
| fieldsAdd content = replacePattern(content, "&lt;&lt;('\"' ('api_key' | 'API_KEY' | 'apikey' | 'APIKEY' | 'key' | 'KEY' | 'token' | 'TOKEN') '\"' ' '* ':' ' '* '\"') [A-Za-z0-9_-]{20,}", "[KEY-MASKED]")
```

The patterns are DPL, not regex; `<<(…)` is a lookbehind, so the key name stays and only the value is replaced — no `$1` back-reference is needed (or supported).

```
// Security health summary
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    security_events = countIf(contains(content, "security") OR contains(content, "auth")),
    failed_events = countIf(contains(content, "failed") OR contains(content, "denied")),
    potential_pii = countIf(contains(content, "@") OR contains(content, "password"))
  }
| fieldsAdd security_event_rate = round((security_events * 100.0) / total_logs, decimals: 2)
| fieldsAdd failure_rate = round((failed_events * 100.0) / total_logs, decimals: 2)
| fieldsAdd pii_risk_rate = round((potential_pii * 100.0) / total_logs, decimals: 2)
```

📝 Summary

In this notebook, you learned:

✅ **Sensitive data discovery** - Finding PII, tokens, and credentials
✅ **OpenPipeline masking** - Configuration and patterns

✅ **IP address analysis** - Classification and masking needs
✅ **Security monitoring** - Auth events, failures, anomalies
✅ **Audit logging** - Admin actions, config changes
✅ **Compliance reporting** - Data access and exposure audits

➡️ Next Steps

Continue to **OPLOGS-99: Best Practice Summary** for the consolidated settings.

🆕 **New Addition (March 2026):** OpenPipeline processes more than logs. To extend these concepts to spans, metrics, and events, continue to **OPIPE-01: OpenPipeline as a Multi-Scope Platform**.

📚 References

- OpenPipeline processing examples (DT docs)
- Set up Grail permissions for logs (DT docs)
- Data privacy and security (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.